《基于事件监听机制的 RBAC 权限管理系统设计与实现》

校园工单管理系统 - RBAC0 与 RBAC1 的实现及 RBAC 模型族分析

课程：系统设计与分析（第三次作业） > 项目形态：Java 17 单机 Swing 应用 >
提示：姓名、学号、班级请按学校要求在提交前补充；本文不虚构个人信息。

1. 作业背景与目标

校园设备报修、场地维护等事务需要由不同人员协同处理。普通人员应能提交和跟踪工单，业务主管需要分派和统计，系统管理员需要维护用户、角色和授权关系。若直接把权限写进界面判断或用户名判断中，授权难以调整，也无法在运行后持久保存。因此，本作业以校园工单为业务场景，构建一个以角色为中介、以数据库为授权事实来源的 RBAC 权限管理系统。

本项目围绕课程要求完成以下目标：

- 在真实程序中实现用户、角色、权限以及多对多关联关系；
- 用 Swing 页面展示不同角色的菜单、页面与按钮能力差异；
- 使用 ActionListener 和 SwingWorker 完成事件响应，避免耗时数据库操作阻塞界面；
- 让 Service 层每次根据可信会话和数据库当前授权进行最终校验；
- 实现 RBAC0，并在此基础上扩展 RBAC1 的角色层次和权限继承；
- 用真实 SQLite、JUnit 与实际 GUI 截图记录测试结果。

系统没有扩展 Web 服务、组织架构、审批流或职责分离等额外业务。RBAC2、RBAC3 仅作为模型分析，不宣称已完成编程实现。

2. RBAC 权限控制原理

RBAC（Role-Based Access Control）把“谁能执行什么操作”拆分为用户（User）、角色（Role）和权限（Permission）。用户与角色之间、角色与权限之间均为多对多关系：同一用户可以承担多个角色，同一角色也可以包含多个权限。用户的有效权限是所拥有角色权限的并集；未被授予的权限默认拒绝。

在本项目中，users 保存登录主体，roles 保存角色定义，permissions 保存能力编码和类别，user_roles 与 role_permissions 保存两组关联。权限编码由程序能力与初始化 SQL 共同定义，但“某个角色拥有哪个编码”只由数据库关联表决定，不能靠用户名或角色名称特判放行。

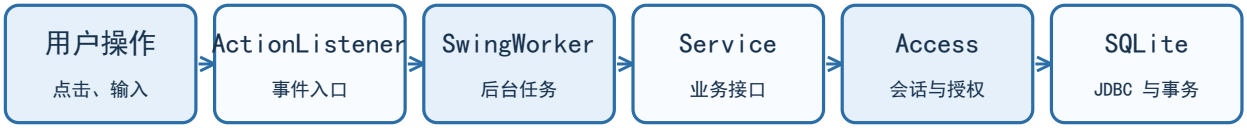
RBAC0：permissions(u) = 并集 permissions(r)，其中 r 属于 roles(u)

多角色并集使一个用户可以同时获得多个业务职责，但不会默认获得数据库中不存在的能力。例如，给只读用户同时分配 VIEWER 和 MANAGER 后，其权限为二者并集；只有 Access 在数据库查询后确认所需编码存在，Service 才会执行操作。

3. 系统总体设计

3.1 技术栈与分层

项目使用 Java 17、Swing、SQLite JDBC、Maven 和 JUnit 5。选择 Swing 的原因是课程要求展示监听器与桌面交互；SQLite 将数据库随应用部署，便于课程演示；Maven Wrapper 保证在未全局安装 Maven 的环境中仍可构建。



UI 呈现能力；Service 与 Access 按数据库当前授权作最终判断。

图 1 系统分层与调用路径

图 1 表示实际分层。ui 负责界面和权限呈现，event 负责后台任务适配，service 定义业务边界，infrastructure 包含数据库事务、会话、密码和统一授权，model 保存不可变数据对象。UI 不直接写 SQL；访问数据库必须经过 Service 与 Access。

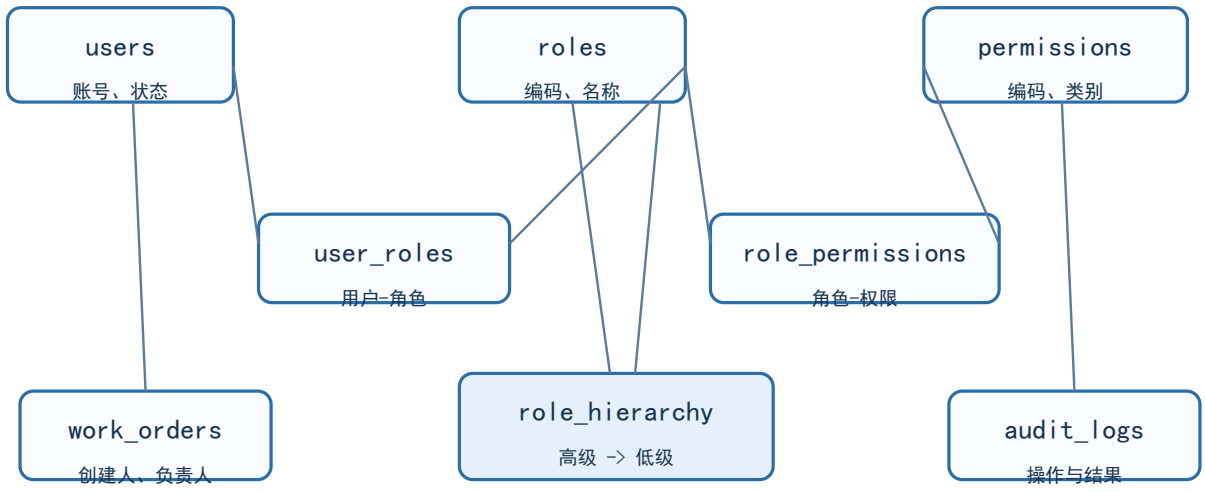
3.2 UI 与 Service 的职责边界

UI 根据当前会话快照决定是否显示菜单、是否允许进入页面、是否禁用按钮，这能让使用者立刻看到能力差异。它不构成安全边界，因为过期界面、伪造调用或绕过按钮均可能发生。最终校验位于 Access.read 和 Access.write：它们从随机不透明会话令牌解析操作人，确认用户仍有效，再查询数据库中的当前有效权限。

写操作在同一 SQLite 写事务中完成授权检查、业务写入和成功审计；权限不足或业务失败时回滚。拒绝记录在回滚后另开事务保存，因此不会因原事务回滚而丢失。该设计把展示逻辑和安全逻辑分开，既能清晰演示界面差异，也能阻止直接调用 Service 的越权请求。

4. 数据库设计与授权持久化

4.1 实体和关联关系



关联表持久化授权；role_hierarchy 是 RBAC1 的版本 2 扩展。

图 2 数据库 ER 图

核心表及用途如下。

表	作用	关键关系或约束
users	用户账号、显示名、密码散列、启用状态	username 唯一；停用后既有会话被拒绝
roles	角色编码与名称	code 唯一
permissions	权限编码、中文名称、类型	类型为 MENU、PAGE、ACTION
user_roles	用户-角色多对多关联	联合主键，角色删除时级联清理
role_permissions	角色-权限多对多关联	联合主键，权限由数据库关系决定
role_hierarchy	高级-低级角色继承关系	版本 2 新增；联合主键、外键、禁止自身继承
work_orders	工单业务数据	创建人与负责人均为用户外键
audit_logs	成功、拒绝与管理操作审计	保留操作人、目标、结果和说明

关联表避免把“多个角色”或“多个权限”重复写入一列，也使管理员能够在运行时调整授权。用户密码使用 PBKDF2-HMAC-SHA256、独立随机盐和迭代参数保存；数据库没有明文演示密码。

4.2 版本 2 迁移

schema_version 记录数据库版本。新数据库先建立基础表和演示数据，再执行 migration_v2.sql；已有版本 1 数据库仅执行迁移。迁移创建 role_hierarchy 和必要索引，并插入版本 2 记录。它不重置 users、密码、工单或 role_permissions，也不预置角色继承关系，因此升级不会意外扩大既有用户的权限。

role_hierarchy(senior_role_id, junior_role_id) 的语义为“高级角色继承低级角色”：MANAGER -> STAFF 表示 MANAGER 取得 STAFF 的有效权限，绝不能反向理解。两列均引用 roles(id)，并使用 ON DELETE CASCADE；删除角色时涉及的继承边自动清理。

5. 监听器与事件响应机制

5.1 删除工单示例

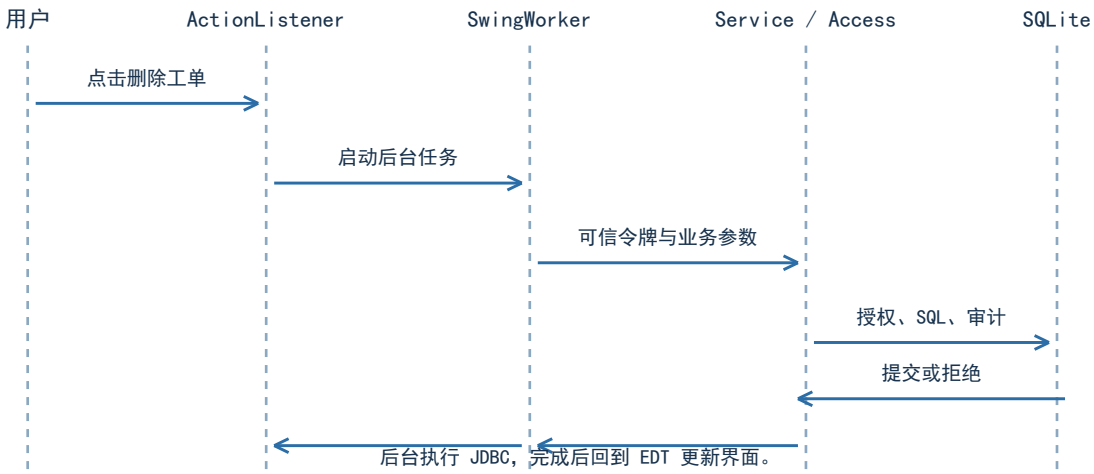


图 3 删除工单的事件流程

以删除工单为例，用户点击按钮后，CampusApplication 中的 ActionListener 先取得当前选中工单并弹出确认框。确认后，BackgroundAction 用 SwingWorker 在后台调用 services.workOrders().delete(token, id)。BusinessServices 声明该操作需要

workorder:delete，Access.write 在数据库写事务中重新计算当前权限；授权通过时删除工单并写入 SUCCESS 审计，否则回滚并独立写入 DENIED 审计。任务结束后才在 EDT 更新工单表格和状态提示。

关键调用形态如下，代码见 CampusApplication.java、BackgroundAction.java、BusinessServices.java 和 Access.java：

```
new BackgroundAction<>(
    () -> { services.workOrders().delete(token, orderId); return true; },
    this::isCurrentSession,
    ignored -> reload.run(),
    this::error
).actionPerformed(event);
```

SwingWorker 的意义在于 JDBC 查询、写事务和密码计算可能耗时；若直接在 EDT 执行，窗口会无响应。后台完成后只能在 EDT 更新 Swing 组件。退出登录会递增界面版本并使旧会话回调失效，避免已退出用户的异步结果覆盖新登录界面。

5.2 四层权限控制差异

控制位置	示例权限	用户可见效果	安全含义
菜单	menu:roles	无权限时不显示“角色与权限”入口	改善导航，不是最终边界
页面	page:roles:view	无权限时不能进入角色页面	阻止常规导航，仍需服务保护
按钮	workorder:delete	删除按钮禁用并显示原因	反馈能力差异，不可信任客户端
Service	Access.write(..., required)	数据库当前权限不足则拒绝	最终安全边界

因此，管理员撤销权限后，即使旧会话尚未刷新界面，下一次 Service 调用也按新数据库状态拒绝；刷新权限后，菜单和按钮同步为新的可见性与可用性状态。

6. RBACO 的实际实现与页面展示

系统初始化四类演示角色。它们是数据库种子数据而不是硬编码放行规则。

角色	典型权限	工单页面效果
ADMIN	全部菜单、页面、用户/角色/授权与工单能力	可进行完整管理
MANAGER	工单新增、编辑、分派、导出、统计	不能删除；不能向拥有系统管理能力的用户分派
STAFF	工单查看、新增、编辑、状态修改	删除、分派、导出不可用
VIEWER	只读查询	仅能查看和搜索工单

下图是实际 GUI 测试生成的 VIEWER 工单页。同一页面保留查询能力，但修改类按钮被禁用，便于课堂上对比角色差异。

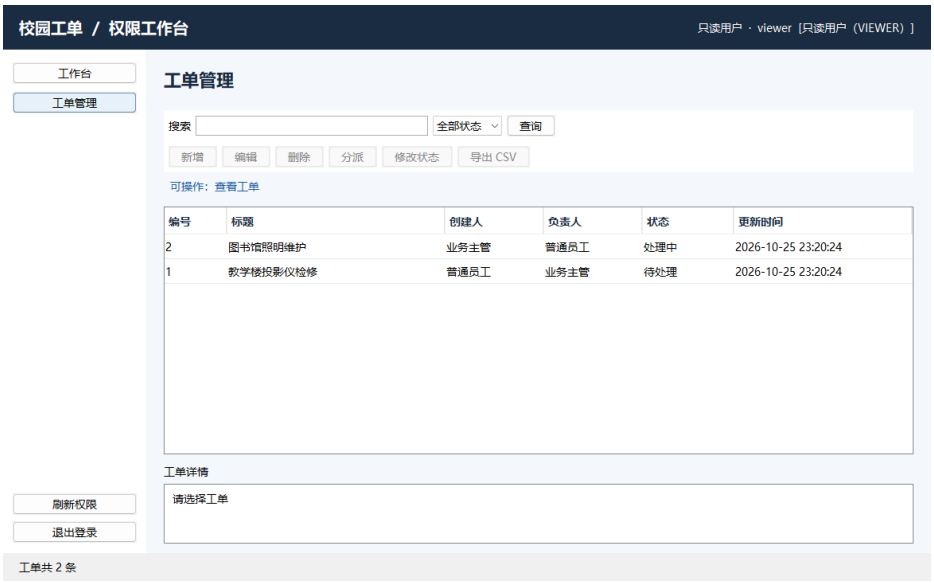


图 4 VIEWER 的工单页面

管理员可在“配置权限”对话框中为某角色调整 MENU、PAGE、ACTION 三类权限，权限编码作为辅助信息保留。授予 STAFF workorder:delete 后，STAFF 刷新权限即可删除真实工单；撤销后，同一既有会话不能继续删除。该过程由真实 SQLite 测试和 GUI 测试共同验证。

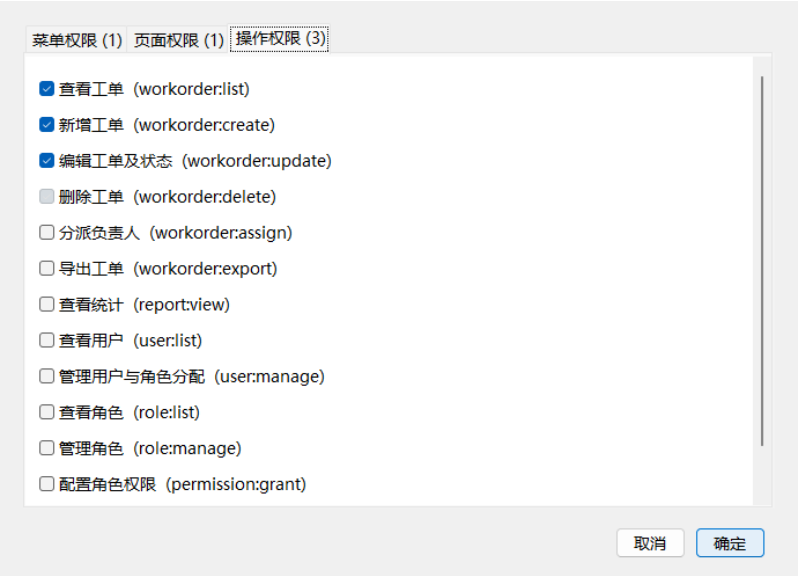


图 5 角色权限配置弹窗

7. RBAC1 的设计与实现

7.1 继承、来源与递归查询

RBAC1 解决的是多个角色存在稳定上下级关系时，重复配置权限的问题。高级角色可继承低级角色的有效权限，但仍可保留自己的直接权限。系统从用户直接角色开始，通过 SQLite 递归 CTE 查询所有可达下级角色，然后对 role_permissions 做 DISTINCT 合并：

```
WITH RECURSIVE reachable(role_id) AS (
    SELECT role_id FROM user_roles WHERE user_id = ?
    UNION
    SELECT h.junior_role_id
    FROM role_hierarchy h JOIN reachable r ON h.senior_role_id = r.role_id
)
SELECT DISTINCT p.code
```

```
FROM permissions p
JOIN role_permissions rp ON rp.permission_id = p.id
JOIN reachable r ON r.role_id = rp.role_id;
```

UNION 与 DISTINCT 保证多条继承路径或多角色同时拥有同一权限时只产生一个有效权限编码。角色页面额外查询直接权限与继承权限，以“直接”“继承”“直接 + 继承”三种来源呈现。若 workorder:delete 同时由父角色直接拥有并由子角色继承，移除继承边后该直接权限仍然存在。

7.2 层次配置与安全约束

管理员在角色页选择角色后，可以看到有效权限总数、直属下级以及按 MENU、PAGE、ACTION 分组的来源。点击“配置继承”会出现明确方向提示：“当前角色作为高级角色，继承下列直属下级角色的有效权限。”界面只调用 Services.Roles，没有把层次关系复制到 UI 中。



图 6 角色权限来源与直属下级



图 7 角色层次配置弹窗

新增、删除或批量替换继承关系均需要 role:manage 和 permission:grant，并在 Access.write 事务中执行。服务层检查角色是否存在、是否自身继承、关系是否重复，以及“拟加入的下级角色是否已可达高级角色”；后者可拒绝多级循环。变更后还要检查至少保留一个启用且具备完整用户管理、角色管理、授权和角色页面入口能力的用户，防止系统失去管理员。

8. RBAC0 与 RBAC1 对比实验

下表总结了 RoleHierarchyTest 在真实 SQLite 和生产 Service 上执行的实验。

实验	操作	实际结果
1：RBAC0 基线	不创建层次关系	role_hierarchy 为空，STAFF 不获得删除权限
2：单向继承	父角色继承子角色，STAFF 被分配父角色	STAFF 的既有会话刷新后获得子角色权限
3：多级继承	高级 -> 中级 -> 低级	高级角色获得中级和低级权限并集
4：来源去重	父角色直接授权且子角色也有同一权限	页面来源为“直接 + 继承”，有效编码只出现一次
5：取消继承	删除父 -> 子关系	仅由继承得到的权限立即消失；直接权限保留
6：循环检测	尝试形成 A -> B -> C -> A	Service 拒绝，事务不留下部分关系
7：持久化与迁移	模拟版本 1 后执行两次初始化	升级到版本 2；工单和已撤销授权保持不变

这些实验同时说明 RBAC1 不是简单的界面显示。每次查询和写操作使用数据库当前关系，因此继承新增、直接撤权、取消继承都会影响已经签发但尚未注销的会话的下一次业务调用。

9. RBAC2、RBAC3 的模型分析

RBAC2 在 RBAC0 基础上增加约束。静态职责分离（SSD）限制某些互斥角色不能同时分配给同一用户，例如“财务申请”和“财务审批”；动态职责分离（DSD）允许用户拥有多个角色，但限制同一会话不能同时激活冲突角色。RBAC2 还可能包含角色基数、前置角色或用户数量上限等约束。若用于本系统，可新增互斥角色对、角色约束和会话激活集合等表，并在分配角色、登录或切换会话时校验。

RBAC3 是 RBAC1 与 RBAC2 的结合：它既有层次继承，又有职责分离等约束。实现时必须额外处理“继承导致的间接冲突”。例如，若 MANAGER 继承 STAFF，而某角色与 STAFF 互斥，则仅检查直接角色不足以发现冲突，必须以递归后的有效角色集合做约束检查。当前项目没有编程实现 RBAC2 或 RBAC3，避免为理论分析加入不属于课程交付范围的业务。

模型	核心能力	本项目状态
RBAC0	用户、角色、权限和两个多对多分配关系	已实现
RBAC1	角色层次、递归权限继承与来源	已实现
RBAC2	SSD、DSD、互斥角色、基数等约束	理论分析，未编程实现
RBAC3	RBAC1 与 RBAC2 的组合	理论分析，未编程实现

10. 测试与运行结果

测试采用 JUnit 5、真实 SQLite JDBC 与生产 Service，不使用权限模拟。最近一次本地验证执行 mvnw.cmd clean verify，共通过 42 项非 GUI 测试：AcceptanceTest 14 项、DatabaseTest 6 项、GuiEventTest 3 项、RoleHierarchyTest 7 项、SecurityTest 12 项，均为 0 失败、0 错误、0 跳过。图形环境中另执行 mvnw.cmd -Pgui-tests test，通过 3 项真实 Swing 测试，并生成本文引用的截图。

测试覆盖首次初始化、版本 1 到版本 2
迁移、权限持久化、伪造与注销会话、动态授权/撤权、工单越权拒绝、管理员保护、审计、并发撤权以及 RBAC1 的多级继承、循环检测、级联清理和权限来源。已生成的 shaded JAR 还通过了独立临时数据库的 --init-only 初始化验证。

GitHub Actions 工作流“Java 17 verification”最近一次公开运行（第 9 次）针对合并后的 RBAC1
代码完成并成功，运行时间为 2026-10-25；本地最终构建仍是本次提交包的直接验证依据。详细命令与结果见 docs/TEST_REPORT.md。

11. 总结

本作业完成了一个能够实际运行的 RBAC 权限管理系统。RBAC0 通过用户-角色-权限关联表、默认拒绝和统一 Service 校验实现基础授权；RBAC1 通过 role_hierarchy、递归查询、来源呈现、环检测和迁移机制实现了可持久化的角色继承。菜单、页面、按钮负责清晰展示差异，Service
与数据库事务负责最终安全保证；ActionListener 与 SwingWorker
使该权限链路在桌面界面中以非阻塞方式执行。

通过本次设计与分析，可以看到权限管理并不只是隐藏一个按钮，而是需要将数据模型、事件响应、会话、业务接口、事务和审计结合起来。RBAC2、RBAC3
的职责分离研究说明了模型继续扩展时需要处理的约束问题，也明确了它们与已实现 RBAC1 的边界。